



Safety Lab

Deployment and Setup Guide

Run Safety Lab Aero on your own infrastructure: your server, your desktop, your AI key. Your data never touches Safety Lab.

safetylabaero.com

Document No.	SL-DG-0001
Title	Safety Lab Aero: Deployment and Setup Guide
Revision	2.1
Date	5 October 2026
Prepared by	Safety Lab Aero
Status	Released

Revision history

Rev	Date	Summary of changes
1.0	7 Sep 2026	Initial release. Self-hosted backend (database, AI proxy, application configuration), defense / ITAR GovCloud variant, browser-only mode, and the isolation checklist.
2.0	14 Sep 2026	Adds the desktop application (install, first launch, pointing it at your server and AI, files-only mode, updates), the two ways to hold an AI key and how to create one, a data-residency overview of the three ways to run the product, and team setup. Restructured into a numbered hierarchy (sections 4 to 7 with subsections).
2.1	5 Oct 2026	Released. Everything since 2.0, which is the revision customers hold. Aligned with SL-EULA-0004 (one acceptance screen on first launch; a customer-hosted install holds no project data, accounts, sign-in events or AI requests at Safety Lab). New section 4.5, the all-in-one option: the database, the sign-in service, the API, the live-update channel and the AI service on one computer the customer owns, installed by one script that needs only Docker, written step by step. The setup file: the install hands every user one file that configures the desktop, loads the license and trusts the server's certificate in one click; no user installs a certificate and IT needs no certificate authority. Section 3 says where every piece comes from, with the download address of the install package. Section 5.3 points at the setup file. Sections 7.2 and 7.3 rewritten for the two-part egress listing (addresses that carry project data and addresses that do not). Section 4.2 documents ALLOWED_ORIGINS; section 6.2 says to create the Anthropic key inside a Workspace. Section 2 names the subprocessor list and the incident process. Setting up the administrator follows the kit of 3 October: on a Supabase project of your own you create and confirm your account before naming it administrator (section 4.1); the all-in-one script creates the administrator account itself, asks for an optional mail server, and without one the administrator adds each user with add-user.sh (section 4.5). The install kit gained the sign-up trigger, the chain-check wrappers, private live-update channels, hardened access rules and lighter version history; an install built from an earlier kit must apply those files, in their numbered order. The server may be addressed by a fixed IP address as well as by a name: the install makes its certificate for the address, and section 4.5 says the address must be static or reserved because the certificate, the license and every setup file are tied to it. On Ubuntu 22.04 or later the install script installs Docker itself. Section 4.5 recommends an Ubuntu server or virtual machine with 16 GB of memory and 50 GB of disk, and names the downloads the install needs (Docker Hub, and Ubuntu's package servers for Docker).

Contents

1 What this sets up.....	4
2 Where your data lives.....	4
3 What you need first.....	5
4 Your server.....	6
4.1 Stand up the database.....	6
4.2 Deploy the AI proxy.....	6
4.3 Host the web application (optional for desktop-only teams).....	7
4.4 Sign in and set up your team.....	7
4.5 The all-in-one option: everything on one computer you own.....	9
5 The desktop application.....	11
5.1 Install it.....	11
5.2 First launch.....	11
5.3 Point the desktop at your server and your AI.....	12
5.4 Files-only alternative.....	13
5.5 Updates.....	13
6 AI keys.....	14
6.1 How AI reaches your model.....	14
6.2 Creating your AI key.....	14
6.3 Per-user keys ("bring your own key").....	14
6.4 Defense and ITAR (GovCloud).....	15
7 Verify and finish.....	16
7.1 Automatic invite emails (optional).....	16
7.2 Verify isolation.....	16
7.3 What "done" looks like.....	16
7.4 Getting help.....	16

1 What this sets up

When you finish, your project data, your sign-ins, and your AI all live on infrastructure you control. Safety Lab holds nothing and is never in the data path.

Safety Lab Aero is delivered as files you host yourself: the application, a small database, a lightweight AI proxy that holds your AI key and checks your license, and a desktop application for each user. This guide walks through standing all of them up on your own infrastructure. Nothing in these steps sends data to Safety Lab, and the application refuses to start if any setting still points back at us. That refusal is deliberate: it is the guardrail that keeps your certification data on your side.

Revision 2.0 adds the desktop application, the two ways to hold an AI key, and a plain description of where every piece of data lives. If you set up with revision 1.0, nothing you did changes; sections 5 and 6 are the new material.

2 Where your data lives

There are three ways to run the product. Most teams use the first, with the desktop application on each user's machine pointed at it.

Way to run it	Where projects live	Sign-in	AI	Best for
Your own server	Your own database (a Supabase project in your organization, or your own PostgreSQL)	Your database's sign-in, your accounts	Your AI proxy, holding your key	A team: shared projects, reviewers, version history
Desktop, files only	Project files on the user's machine, or your SharePoint / OneDrive	None needed; the license covers the machine	Your AI proxy, or off	A single engineer, or an air-gapped machine
Browser only	The user's browser storage on that machine	None	Off unless the user supplies a key	Evaluation on one machine, no install

In every case the answer to "what does Safety Lab hold" is the same: nothing. No project, no document, no account, no sign-in event, and no AI request passes through or is stored on a Safety Lab system. The application's network guard enforces this: the desktop application can only reach the addresses you configure, and the web application refuses to start if any address points at Safety Lab.

One more piece of data is worth naming, because it is the only thing the application keeps outside your project: a drafting correction store. When you edit a sentence the AI drafted, the application records what it drafted and what you changed it to, so later drafts match how you write. The store lives in your own browser profile on your own machine. It is never sent to Safety Lab, it never serves another customer, and short excerpts from it ride along only in your own AI requests, which go to your own proxy and your own provider. It is excluded from export-controlled work in both directions: nothing captured under a controlled project is ever reused, and nothing is reused at all while a controlled project is open. You can count it, cap it, set the cap to zero, or clear it from the AI settings at any time.

The AI never trains on your content, on any path. Your corrections shape later drafts by being quoted back to the model in your own requests, not by changing any model. Section 5 of the End User License and Subscription Agreement states this as a contractual commitment.

The desktop application and the web application are two views of the same thing. A team on its own server can use either or both: sign in once, see the same projects, edit in either, and open a project from the web in the desktop or the other way around.

Two further documents answer the questions this section raises, and both are part of the Documentation the license agreement refers to. **SL-SUB-0001 Subprocessor List** names every third party that processes customer data, what each receives, and where, and it states which of them are in the path at all on a self-hosted install. **SL-IRP-0001 Security Incident and Breach Notification Process** sets out how an incident is found, who decides, and the seventy-two hour notification. Ask Safety Lab for the current revision of either.

3 What you need first

- **A database you control.** The simplest is a Supabase project in your own organization; it also provides the sign-in and the live updates the application uses. Or the all-in-one option of section 4.5, which runs those same services on a computer you own. A bare PostgreSQL server on its own does not work: the application talks to the sign-in and API layer in front of the database, not to the database directly.
- **One AI option.** Your own Anthropic (Claude) API key, or Azure OpenAI, or for defense programs AWS Bedrock GovCloud or Azure OpenAI Government (section 6.4).
- **Somewhere to run the AI proxy.** A Cloudflare account, or any host that runs Node 18 or later: a container or virtual machine in your own cloud.
- **The install package.** One zip file, about half a megabyte, downloaded from <https://updates.safetylabaero.com/customer-install/SafetyLabAero-customer-install-2026-10-05.zip>. It holds the database kit, the AI proxy, the all-in-one installer of section 4.5 and the setup tools. It contains no secrets and nothing of yours; everything personal is generated on your machine when you run it.
- **The desktop application.** Your users download it themselves from <https://safetylabaero.com> (Mac and Windows). It is the same download for every customer; what makes it yours is the server address and the license file each user enters.
- **Docker.** Only for the all-in-one option (section 4.5). On an Ubuntu 22.04 or later server the install script installs it for you. On Windows or Mac, Docker Desktop from <https://www.docker.com/products/docker-desktop/>; on another Linux server, Docker Engine and its compose plugin from your distribution. It is the one program needed; everything else runs inside it.
- **Node.js and Git.** Not needed for the all-in-one option. Node 18 or later from <https://nodejs.org> only if you run the AI proxy yourself as a plain service (section 4.2, option B) or host the web application (section 4.3). Git is never needed.
- **Your license file and public key.** Your license (a small file ending in .lic) and the public-key list used to check it. The public key is safe to share and is not a secret.

If your users only ever work in the desktop application, you can skip hosting the web application (section 4.3); everything else in section 4 still applies.

4 Your server

4.1 Stand up the database

From the database folder of the delivered kit, run the installer against your database connection string. In Supabase this is under Project Settings, Database, Connection string.

```
./apply.sh "postgresql://postgres:PASSWORD@db.YOURREF.supabase.co:5432/postgres"
```

Then make yourself an administrator. First create your account in the application with that address (point the application at this database as in section 4.3 or 5.3) and confirm it from the email it sends. Keep Confirm email switched on in your Supabase project's Auth settings; it is on by default. Only then, in the Supabase SQL editor or with `psql`:

```
insert into private.platform_admins(email) values ('you@yourcompany.com');
```

Do it in that order: the administrator role follows the address, so it must belong to an account whose address you have confirmed yourself. Your database is ready. The install starts with an empty administrator list, you just added the first entry, and carries none of Safety Lab's own settings. To confirm the database is healthy without opening the application, run the checker from the kit against the same connection string:

```
./verify.sh "postgresql://postgres:PASSWORD@db.YOURREF.supabase.co:5432/postgres"
```

4.2 Deploy the AI proxy

The proxy is the piece that holds your AI key. Every AI request from the application goes to the proxy, the proxy checks your signed license, and only then forwards the request to your AI provider using your key. No user's machine ever holds the key, and your provider bills you directly for your own usage. Both ways of running it use the same three settings:

- **LICENSE_MODE** set to offline.
- **LICENSE_PUBLIC_KEYS**, the public-key list Safety Lab gave you (a short JSON array).
- **ANTHROPIC_API_KEY**, your own Claude key (section 6.2 explains how to create one). For Azure OpenAI, use the Azure settings in the proxy folder instead.
- **ALLOWED_ORIGINS**, the web address your application is served from, for example `https://app.yourcompany.com`. The proxy then accepts browser requests only from there. Leave it unset and the proxy says so in its log and accepts any origin, which is deliberate for an install that has no browser front end, but set it if you host the web application.

Option A. On Cloudflare

From the proxy folder:

```
npx wrangler secret put LICENSE_PUBLIC_KEYS
npx wrangler secret put ANTHROPIC_API_KEY
npx wrangler deploy
```

Set `LICENSE_MODE=offline` as a plain variable in the proxy configuration, then note the address Cloudflare gives you (for example `https://your-proxy.workers.dev`).

Option B. As a standalone service in your own cloud

Anywhere that runs Node 18 or later, a container or virtual machine inside your own network:

```
cp .env.example .env      # set LICENSE_MODE, LICENSE_PUBLIC_KEYS, ANTHROPIC_API_KEY
node server.mjs           # listens on :8787
```

Put it behind your normal HTTPS front door and note that address. Confirm it is alive: a request to `/v1/ai/health` returns `{"ok":true}`. Write the proxy address down; you will enter it in the web configuration (section 4.3) and in every desktop application's settings (section 5.3).

4.3 Host the web application (optional for desktop-only teams)

Serve the application files from your own web host. The application needs a short configuration script before its own scripts. The easiest way is to generate it: copy `install.env.example` to `install.env` in the delivered kit, fill in your addresses, and run the generator.

```
cp install.env.example install.env  # then edit install.env
node configure.js                   # writes slab_env.js
```

Include the `slab_env.js` it produces before the application's own scripts. The generator refuses to write a file that points anything at a Safety Lab address, and it catches the one easy mistake, leaving the AI endpoint blank, which would otherwise be an error at startup. If you prefer to write it by hand, add this instead, filling in your addresses:

```
<script>
  window.__SLAB_SUPABASE_URL__ = "https://YOURREF.supabase.co";
  window.__SLAB_SUPABASE_KEY__ = "YOUR_SUPABASE_ANON_KEY"; // publishable key
  window.__SLAB_AI_ENDPOINT__ = "https://your-proxy-address";
  window.__SLAB_WEB_APP_URL__ = "https://app.yourcompany.com"; // this install
</script>
```

If any of these still points at a Safety Lab address, the application refuses to start and names the setting.

4.4 Sign in and set up your team

Open the application (web or desktop). On first run it asks for your license file; load it. Sign in with the account you made in section 4.1. Then:

1. **Create a workspace.** Whoever creates it is its owner. Nobody joins a workspace without an invitation.
2. **Invite your team.** From the workspace, invite each member by email. Without the optional invite-email function (section 7.1) the application shows an invitation link for you to pass along by hand; with it, the invitation is emailed automatically.
3. **Set roles.** Owners and administrators manage membership; editors work on projects; viewers are read-only reviewers. Only the owner can transfer ownership, and administrators cannot remove the owner.
4. **Make a test project** and confirm it appears in your own database tables, not ours, and that AI drafting works, going through your proxy to your AI.

4.5 The all-in-one option: everything on one computer you own

Sections 4.1 to 4.4 assume a Supabase project in your own cloud account. This section is the other way: the database, the sign-in service, the API, the live-update channel and the AI service all run on one computer inside your building, inside Docker, with no cloud account at all. Once installed, the only outside address it talks to is your AI provider, for drafting requests, and those never carry your project files. One script does the whole install, and it asks a few questions. It is written below for someone who has never done this before; if you have, skim the headings.

What the script does for you: copies the stock Supabase self-hosting files (release self-hosted/v0.8.2, included in the package, so nothing is fetched from GitHub), generates every password and key for this installation, puts https in front with your certificate, starts the services, builds the Safety Lab database from the kit, checks the sign-in service and the AI service answer over https, and writes a one-page sheet of what every user types in the application. Two stock services are switched off because the application never uses them: the connection pooler, the only thing that would expose the database port outside the computer, and the edge-function runtime, which downloads packages from the internet when it starts and so would never come up on an air-gapped machine.

Part 1. Get the computer ready (your IT person, about 20 minutes)

1. **Pick the computer.** It must stay switched on. The simplest is an Ubuntu 22.04 or 24.04 server or virtual machine; Windows 10 or 11 Pro or a Mac also work. At least 4 processor cores, 16 GB of memory (8 GB is the minimum and runs tight) and 50 GB of free disk.
2. **Install Docker Desktop (Windows or Mac only).** On Ubuntu skip this step and the next: the script installs Docker itself. Go to <https://www.docker.com/products/docker-desktop/> and click Download. Run the installer and accept the defaults. When it asks to use WSL 2 on Windows, say yes. Restart if it asks.
3. **Open Docker Desktop once (Windows or Mac only).** Wait until the bottom left corner says Engine running. Leave it open; it starts by itself from now on.
4. **Give the computer a name on your network,** for example `safetylab.yourcompany.local`. Your IT person does this in your DNS. Test from another computer: open a command window and type `ping safetylab.yourcompany.local`. It should answer. No name? Use the computer's IP address instead, for example `10.20.30.40`. It must be fixed: ask IT to make it static or reserve it. The certificate, your license and every user's setup file are tied to the name or address, so changing it later means a new license and a new setup file for everyone.
5. **Make the AI key.** Go to <https://console.anthropic.com> and sign in (make an account if you have none). Click Settings, then Workspaces, then Create Workspace, and name it Safety Lab. Open that workspace, click API Keys, Create Key, name it `safetylab-server`, and copy the key somewhere safe on this computer only.
6. **Check the computer can open <https://api.anthropic.com> in a browser.** It shows a short message; that is fine. Once installed, it is the only outside address the computer needs. During the install it also downloads the software, about 9 GB, from Docker Hub (`hub.docker.com` and its download servers) and, on Ubuntu, Docker itself from Ubuntu's package servers. Ask IT to allow those for the install.

You do not need a certificate. The script makes one, and the desktop application trusts it for this server automatically through the setup file, so no user installs anything. If your IT would rather use a certificate from

the company's own authority, have the two files (certificate and private key, PEM) ready and give them when the script asks; everything else is the same. With an IP address, use the script's own certificate.

Part 2. Install (about 20 minutes, most of it waiting)

1. **Unzip the install package** (section 3) into C:\SafetyLab on Windows, or your home folder on a Mac or Linux. You get a folder named customer-install. If Safety Lab sent the package after you told them the server name or IP address, your license file is already inside it.
2. **Windows: open the folder C:\SafetyLab\customer-install\selfhost and double-click install.cmd.** It opens the Ubuntu window that Docker Desktop set up and starts the install for you. If it says WSL is not installed, follow the two lines it prints, then double-click it again. Mac or Linux: open the Terminal app and type the two lines below, pressing Enter after each.

```
cd ~/customer-install/selfhost
bash install.sh
```

3. **Answer the questions**, pressing Enter after each: the server name or IP address from Part 1 step 4; press Enter twice at the certificate questions (unless IT gave you files); the AI key from step 5 (nothing shows while you type, which is normal); press Enter at the Workspace ID question unless Safety Lab told you otherwise; the administrator's email address (yours, if you look after this server); then your mail server's name, port, user name, password and sender address if IT gave you those, or press Enter to skip if you have no mail server (Part 4 explains what that changes).
4. **Wait.** The first run downloads about 9 GB. You see the database files being applied one by one, then Checking the front door, then two lines that should both say 200, then Done and a short sheet. If this run created the administrator account, the last lines show its email and a temporary password. They are shown this once and saved nowhere, so write the password down now.
5. **Two files now sit next to install.sh:** the setup file, named after your server and ending in .safetylab-setup, which is what every user needs, and WHAT-TO-TYPE-IN-THE-APP.txt, which explains it and lists the values in case anyone wants to type them by hand.
6. **Only if your license file was not in the package:** when it arrives (it ends in .lic), put it next to install.sh and run the install again. The setup file is rewritten with the license inside.

If something goes wrong the script stops and prints a sentence starting with STOP that says what to do. Running bash install.sh again is always safe: it remembers your answers (not the AI key, which lives only in stack/ai-proxy.env) and never deletes your data.

Part 3. Every user's computer (two minutes each)

1. **Download the desktop application** from <https://safetylabaero.com> (Mac or Windows) and install it like any application.
2. **Open it and click Choose setup file**, then pick the .safetylab-setup file your administrator gave you, or drop the file on the window. It fills in the server, the AI endpoint, the license and the server's certificate; nothing to type and nothing to install. Click Next, read and accept the agreement, click Finish. (If the setup file carried no license yet, the same screen takes the .lic file on its own.)
3. **Sign in.** If your server has a mail server, click Create account, use your work email, and click the link in the email you receive. If it does not, your administrator creates your account and gives you a temporary password: sign in with it, then change it under Account, Change password.

Part 4. The administrator account

The install script asks for the administrator's email, creates that account itself and makes it the administrator, then prints its temporary password once at the end of the install (Part 2). Sign in with it and change it under Account, Change password. To add a user on a server with no mail server, run `./add-user.sh person@yourcompany.com` in the selfhost folder and give them the password it prints. Why: without a mail server nobody can check that a person owns the address they type, and the application trusts that address for invitations, the administrator role and sign-off records, so open sign-up is off on such a server.

Day to day

Is it running? In the stack folder: `docker compose ps`. Every line should say healthy or Up. Stop: `docker compose stop`. Start: `docker compose start`. After a reboot it starts by itself as long as Docker starts (Docker Desktop on Windows or a Mac; on Ubuntu, Docker starts with the machine).

Back up these three things somewhere safe, regularly: the folder `stack/volumes/db/data` (your data), and the files `stack/.env` and `stack/ai-proxy.env` (the keys). Losing the keys with the data intact means nobody can sign in.

Updates. Database updates arrive as new numbered files for the db folder; copy them in and apply each new one in order with the command in the package README. A new package version is unzipped over the old one, keeping `selfhost/stack` and `selfhost/answers.env`, and `bash install.sh` is run again.

5 The desktop application

5.1 Install it

The desktop application is the same product in a window of its own: it opens project files, works offline, and syncs with your server when it is back online. Safety Lab provides installers for each platform.

macOS

Two installers are provided, one for Apple silicon (arm64) and one for Intel (x64). Open the .dmg and drag Safety Lab Aero to Applications. On the first launch, right-click the application and choose Open, then confirm. macOS shows a warning the first time because the current build is not yet notarized with Apple; the right-click step is only needed once. Later builds will be notarized, at which point the warning goes away.

Windows

Run the installer provided by Safety Lab and follow the prompts. Windows may show a SmartScreen notice the first time; choose More info, then Run anyway. As with macOS, this is because the build is not yet signed with a publisher certificate, and it is a one-time step.

What the application can reach

The desktop application is fenced at the network layer. It can only contact the addresses you enter in its settings: your server and your AI proxy. It cannot reach Safety Lab, fonts, analytics, or anything else, even if a script tried. There is nothing to configure for this; it follows from your settings.

5.2 First launch

The first time the application opens it walks through three short screens before the main window appears:

1. **License.** Choose your .lic file, or paste its contents. It is checked on this computer and never sent anywhere. Trials use a temporary license with an end date; the screen shows what the license covers and when it ends.
2. **End User License and Subscription Agreement.** Read and accept. This is one agreement covering both the license and the subscription. Earlier versions of the product asked you to accept two separate documents; they were merged in September 2026.
3. **Screen lock (optional).** A passcode adds a lock screen when the application opens. It is a screen lock, not an identity: you sign in to your account inside the application exactly as on the web, and stay signed in on this computer. Leave it blank for no lock.

After this the application opens against the Safety Lab trial cloud by default. For a team on its own server, the next step is to point it at your server.

5.3 Point the desktop at your server and your AI

If your organization gave you a setup file (it ends in .safetylab-setup), you do not type any of this: Choose setup file on the first screen, or the same button under Settings, fills in every field below and the license in one go. The all-in-one install of section 4.5 writes that file. The values are also listed in WHAT-TO-TYPE-IN-THE-APP.txt next to it, for anyone who prefers to type them.

Open Settings: on macOS it is under the Safety Lab Aero menu (or Cmd+comma); on Windows it is under File (or Ctrl+comma). Settings are stored on that computer only.

Where your data lives

Choose **My organization's server** and enter three things:

- **Server address.** Your database's address, for example <https://YOURREF.supabase.co>.
- **Server key (publishable).** The publishable, or anon, key from your Supabase project. This key is designed to be distributed to clients; it is not the secret service key.
- **Web address.** Only if you host the web application (section 4.3): its address, so "Open this project on the web" works. Leave blank otherwise.

AI

Choose **My organization's AI endpoint** and enter your proxy address from section 4.2, followed by /v1/ai, for example <https://your-proxy.workers.dev/v1/ai>. If your team does not use AI features, choose Off; everything else works.

Click Save and reload. The application checks the settings before it restarts and refuses, in plain words, if anything still points at Safety Lab or if a required address is missing. Then sign in with your account. You stay signed in on this computer.

Confirm it

- Open a project you created on the web. It should appear in the desktop, and an edit made in one should appear in the other.
- Run an AI draft. It goes through your proxy to your AI; nothing reaches Safety Lab.
- The License card in Settings shows your organization, tier, seats, and end date.

5.4 Files-only alternative

A single engineer, or a machine that must not talk to any server, can run the desktop application with no backend at all. In Settings choose **Files only**. Projects are then files you open and save from the File menu (Open Project File, Save Project File, Save As). Keep them anywhere you like, including a folder synced to your own SharePoint or OneDrive. AI can still run through your organization's proxy if you enter its address, or be off. There is no sign-in in this mode; the license covers the machine.

5.5 Updates

Updates are never installed silently. Use Help, Check for Updates (on macOS also under the Safety Lab Aero menu). If a newer version exists the application tells you and opens the download page; you install it the same way as the first time. Every update announcement is signed by Safety Lab and the application verifies that signature before believing it, so a tampered download page cannot push you a build we did not publish.

6 AI keys

6.1 How AI reaches your model

Every AI request from the application, web or desktop, goes to your proxy. The proxy holds your provider key, checks your signed license, and forwards the request to your AI provider. Three things follow from this design.

- **No user's machine holds the key.** Users see only the proxy address. Rotating the key is a change in one place.
- **Your provider bills you.** Usage runs on your own account with your provider. Safety Lab does not meter, cap, or see it.
- **Controlled work stays inside your boundary.** Projects marked export-controlled are only sent to the sovereign backend you configure (section 6.4); if none is configured the request is refused, never sent to a public model.

The AI never trains on your content, on any path (section 2). Your provider does not retain your prompts or outputs for its own training either, and section 5 of the End User License and Subscription Agreement commits Safety Lab to the same, in every way the product can be deployed.

6.2 Creating your AI key

Anthropic (Claude)

1. Sign in to the Anthropic Console (console.anthropic.com) with an account that belongs to your organization, not an individual's personal account.
2. Create the key inside a Workspace rather than at the organization level, and name it for this purpose, for example "Safety Lab Aero proxy", so it can be rotated on its own later. A key created inside a Workspace needs nothing further. If you use an organization-level key instead, Anthropic requires the request to name a workspace, so set `ANTHROPIC_WORKSPACE_ID` in the proxy configuration to that workspace id; it is not a secret.
3. Set a monthly spending limit on the organization. The proxy has no cap of its own, so this is your budget control.
4. Copy the key once, into the proxy configuration (section 4.2) and nowhere else. Anthropic does not show it again.

Azure OpenAI

Create an Azure OpenAI resource in your subscription, deploy a model, and note the endpoint, deployment name, and key. Enter them in the proxy's Azure settings (the proxy folder documents the variable names). For defense programs use the Azure Government variant in section 6.4.

6.3 Per-user keys ("bring your own key")

The application also has an Advanced setting where a user can paste an Anthropic key of their own, which sends requests directly from that machine to Anthropic. It is useful on the Safety Lab trial cloud, where a user evaluating the product wants to use their own key without any setup.

On your own server, and in the desktop application, this path is not used, and that is by design. The desktop application can only reach the addresses you configured (section 5.1), and the web application on your server routes AI through your proxy. Both mean a personal key pasted into the application will not reach Anthropic from those installs. For a team we recommend the proxy for exactly this reason: one key, held in one place, with the license check and the export-control fence in front of it. Users who evaluated the product on the trial cloud with a personal key can remove it under Advanced AI settings once they move to your server.

6.4 Defense and ITAR (GovCloud)

Same as section 4, but run the proxy as the standalone service (Option B in section 4.2) inside your own AWS GovCloud network, and point it at a US-sovereign AI instead of a public key. In the proxy's environment set one of the following.

AWS Bedrock GovCloud (Claude, the engine the product was validated on)

```
ITAR_PROVIDER=bedrock
AWS_BEDROCK_REGION=us-gov-west-1
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_BEDROCK_MODEL_ID=anthropic.claude-3-5-sonnet-20240620-v1:0
```

Azure OpenAI Government

Set ITAR_PROVIDER=azure and the Azure variables (see the ITAR provisioning notes in the proxy folder).

If no sovereign backend is configured, a controlled request is refused, never sent to a public model. Mark controlled projects as export-controlled in the application; the data fence and this proxy both enforce it.

7 Verify and finish

7.1 Automatic invite emails (optional)

Not required. Without it, when an administrator invites a teammate the application shows a link to pass along by hand. If you want invitations emailed automatically, deploy the one invite-email function to your database with your own email-provider key. It is the only server-side function a self-hosted install can use, and it is optional.

7.2 Verify isolation

In the browser console of the web application, or in the desktop application's View menu with developer tools enabled by Safety Lab, run **SLConfigEgress()**. It lists every address the application will contact, and says of each one whether it carries project data. Every address that carries project data should be yours: your database, your proxy. If one of those shows a Safety Lab address, correct the matching setting. The desktop application also lists one address that carries no project data, `updates.safetylabaero.com`, which is the update check described in section 5.5; it sends the installed version and the platform and nothing else, and it can be turned off. Seeing it is not a finding. On the database side, `verify.sh` (section 4.1) checks the tables, security rules, and administrator account are in place and prints a pass or fail summary.

7.3 What "done" looks like

- Your database holds your projects; ours holds nothing.
- Your proxy holds your AI key; no user's machine ever sees it; your provider bills you; controlled work never leaves your boundary.
- Each desktop application is pointed at your server and your proxy, and can reach nothing else.
- `SLConfigEgress()` shows only your own addresses carrying project data, and nothing of ours among them. On the desktop the update-check address appears separately, marked as carrying no project data.
- Your team is invited, roles are set, and a test project round-trips between web and desktop.

7.4 Getting help

Your onboarding session with Safety Lab covers this guide end to end on your own infrastructure, remote or on site. For anything after that, contact your Safety Lab representative; support is not capped.